

GUÍA DE TRABAJO

N.º 03 Funciones

Profesor: Guillermo Cid Ampuero

Indicaciones generales

- Resuelva cada ejercicio en un archivo independiente utilizando un formato de nombre claro, por ejemplo: `ejercicio_1.py`, `ejercicio_2.py`, etc.
- Mantenga el estándar de importación de librerías: utilice `import numpy as np` e `import matplotlib.pyplot as plt` cada vez que sean necesarias.
- Cuando el objetivo del problema sea implementar un método numérico desde cero, no utilice funciones integradas de Python (como las de `scipy`) que entreguen la solución de forma directa, a menos que se solicite explícitamente para comparar resultados.
- Priorice la claridad en su código. Utilice comentarios para explicar la lógica de su solución, defina funciones propias cuando sea pertinente y nombre sus variables de forma descriptiva.
- En todos los gráficos generados incluya obligatoriamente: título, nombre de los ejes, unidades de medida, cuadrícula (*grid*) y, cuando exista más de una curva, una leyenda.
- Valide sus resultados: antes de ejecutar un programa, analice el problema e intente anticipar el orden de magnitud o el comportamiento esperado de la solución.

Ejercicios

Ejercicio 1 – Calculadora básica con funciones

Se desea construir un programa que simule una calculadora básica utilizando funciones independientes para cada operación matemática.

El programa debe realizar lo siguiente:

- Defina tres funciones separadas: `suma(a, b)`, `multiplicacion(a, b)` y `division(a, b)`. Cada una debe recibir dos números y retornar el resultado de la operación.
- En la función de división, incluya un condicional `if` para evitar la división por cero. Si el denominador es cero, debe retornar un mensaje de error (por ejemplo, `None` o un `string`).
- Defina una función principal llamada `calculadora(operacion, a, b)`.
- Dentro de `calculadora`, utilice una estructura `if-elif-else` que evalúe el parámetro `operacion` (un `string` como "sumar", "multiplicar" o "dividir") y llame a la función correspondiente, retornando su valor.
- Pruebe su programa llamando a la función `calculadora` con la operación "multiplicar" los números 5 y 4. Imprima el resultado en pantalla.

Ejercicio 2 – Velocidad en caída libre

Un objeto se deja caer libremente o se lanza hacia abajo. Su velocidad en función del tiempo está dada por la ecuación:

$$v(t) = v_0 - g \cdot t$$

donde v_0 es la velocidad inicial, $g = 9.81 \text{ m/s}^2$ y t es el tiempo.

Escriba un programa que:

- Importe `numpy` y `matplotlib.pyplot`.
- Defina una función llamada `velocidad_caida_libre(t, v0, g)` que reciba estos tres parámetros y retorne la velocidad evaluada en t .
- Cree un arreglo de tiempo utilizando `np.linspace()` con 50 valores, empezando en 0 hasta 5 segundos.
- Llame a su función pasándole el arreglo de tiempo, una velocidad inicial $v_0 = 0$ y $g = 9.81$. Guarde el resultado en una variable `velocidades`.
- Grafique la velocidad en función del tiempo. Incluya título, etiquetas en los ejes, cuadrícula y unidades.

Ejercicio 3 – Trayectoria de un balón de fútbol paso a paso

A diferencia del ejercicio anterior, en este caso calcularemos la posición punto a punto utilizando un ciclo. La posición (x, y) en un instante t está dada por:

$$x = v_0 \cos(\theta)t, \quad y = v_0 \sin(\theta)t - \frac{1}{2}gt^2$$

- Defina una función `posicion_balon(t, v0, angulo_rad, g)` que reciba un valor **escalar** de tiempo y retorne **dos** valores: la posición x e y en ese instante.
- Defina los parámetros $v_0 = 25 \text{ m/s}$, $\theta = 30^\circ$ (conviértalo a radianes) y $g = 9.81 \text{ m/s}^2$.
- Utilice `np.arange()` para crear los tiempos de 0 a 3 segundos en pasos de 0.05.
- Cree dos listas vacías: `x_lista` e `y_lista`.
- Utilice un ciclo `for` para iterar sobre el arreglo de tiempos. En cada iteración, llame a su función para obtener x e y .
- Use un condicional `if` para añadir (`append`) los valores a las listas **solo si** la altura $y \geq 0$.
- Grafique la trayectoria utilizando las listas resultantes, incorporando todos los elementos de diseño del gráfico solicitados.

Ejercicio 4 – Procesamiento de sensores con `*args`

Es común recibir una cantidad variable de lecturas de sensores de terreno. El operador `*args` en Python permite a una función recibir un número indeterminado de argumentos posicionales.

- a) Defina una función llamada `procesar_sensores(*args)`.
- b) Dentro de la función, verifique si se ingresaron argumentos. Si no hay datos, retorne una lista vacía mediante un condicional `if`.
- c) La función debe iterar sobre `args` con un ciclo `for`, validando mediante un condicional `if` que los valores sean mayores o iguales a 0 (se asume que un valor negativo es un error del sensor).
- d) Almacene los valores válidos en una lista nueva y retórnela.
- e) Llame a la función pasando los siguientes valores sueltos (no como lista, sino como argumentos múltiples): 15.2, 16.4, -3.1, 14.8, -0.5, 17.1.
- f) Guarde el resultado devuelto en una lista llamada `datos_limpios` y grafique estos puntos mediante un gráfico de dispersión (utilizando un marcador como `'o'`).

Ejercicio 5 – Saturación de un amplificador (Clipping)

Un amplificador de audio no puede entregar más voltaje que el de su fuente de alimentación. Si la señal de entrada multiplicada por la ganancia supera este límite, la señal se recorta (clipping).

- a) Defina una función `amplificar_senal(senal_entrada, ganancia, vmax)`.
- b) Dentro de la función, cree una lista vacía `senal_salida`.
- c) Utilice un ciclo `for` para recorrer la `senal_entrada`. Para cada punto, calcule $v_{\text{amp}} = v_{\text{in}} \times \text{ganancia}$.
- d) Utilice un bloque `if-elif-else` para limitar el voltaje:
 - Si $v_{\text{amp}} > v_{\text{max}}$, asigne v_{max} .
 - Si $v_{\text{amp}} < -v_{\text{max}}$, asigne $-v_{\text{max}}$.
 - De lo contrario, asigne el valor sin modificar.
- e) Añada el resultado a la lista de salida y retorne dicha lista.
- f) En su programa principal, cree un arreglo de tiempo de 0 a 0.1 s y genere una señal de entrada $v_{\text{in}} = 2 \sin(2\pi \cdot 20 \cdot t)$.
- g) Llame a su función con una `ganancia=4` y `vmax=5`.
- h) Grafique en una misma figura la señal amplificada (con clipping) y la señal de entrada. Incluya la leyenda correspondiente.

Ejercicio 6 – Control On/Off de un Termostato

Se desea simular el comportamiento de la temperatura T de una habitación controlada por un calefactor. La lógica de control es simple:

- Si $T < T_{\text{objetivo}} - 1^{\circ}\text{C}$, el calefactor se enciende (estado = 1).
- Si $T > T_{\text{objetivo}} + 1^{\circ}\text{C}$, el calefactor se apaga (estado = 0).
- En cualquier otro caso, el calefactor mantiene su estado anterior.

La temperatura se actualiza según la regla (método iterativo):

$$T_{\text{nuevo}} = T_{\text{anterior}} + \Delta t \cdot (1.5 \cdot \text{estado} - 0.2 \cdot (T_{\text{anterior}} - 15))$$

(donde 15°C es la temperatura exterior).

- a) Defina una función `simular_termostato(T_inicial, T_objetivo)`.
- b) Inicialice listas para almacenar la temperatura y los tiempos, comenzando desde el tiempo 0 y con un estado inicial del calefactor apagado (0).
- c) Utilice un ciclo `for` para iterar el tiempo de 0 a 30 minutos con pasos de $\Delta t = 0.5$ minutos.
- d) Use condicionales `if-elif` para determinar el estado del calefactor en cada iteración según la regla descrita.
- e) Calcule la nueva temperatura, agréguela a su respectiva lista y así también el instante de tiempo. La función debe retornar ambas listas.
- f) Grafique la temperatura simulada y trace una línea horizontal (puede usar `plt.axhline`) para mostrar la T_{objetivo} .

Consejos

- ejecute cada programa desde el inicio;
- compruebe que no existan errores de ejecución;
- revise que los resultados sean coherentes;
- verifique las unidades utilizadas;
- compruebe que todos los gráficos tengan título, nombres de los ejes y cuadrícula.